

REX Deep Research Agent — Technical Expo & Viva Guide

Comprehensive Architecture, Algorithms, Models, and Technical Q&A; Preparation
System Version: 2.0.0 · Generated: July 29, 2026

1. Executive Summary & Elevator Pitch

REX (Recursive Exploration eXplorer) is a self-improving multi-agent deep research framework engineered to automate comprehensive technical, market, and academic intelligence gathering. Unlike standard single-prompt LLM interactions, REX executes multi-step recursive investigation across live web and vector database indices, filters low-credibility fluff, detects analytical evidence gaps, and produces fully cited research reports accompanied by real-time quality evaluation telemetry.

2. 9-Stage Workflow & Component Breakdown

Stage	Agent Node	Input / Output Data	Technical Responsibility
1. Plan	Planner	Query → `sub_questions`	Classifies topic category & decomposes query into 4-8 sub-questions.
2. Recall	Memory Retrieval	Query → `retrieved_memory`	Performs vector embedding search over past research reflections.
3. Search	Searcher	Questions → `raw_pages`	Executes parallel scrapers (DuckDuckGo, Wikipedia, arXiv, Playwright).
4. Filter	Filter & Analyzer	Raw pages → `scored_chunks`	Applies domain blocklists, chunks text, and ranks vector cosine relevance.
5. Synthesize	Synthesis	Chunks → `synthesis_results`	Synthesizes evidence snippets into structured section drafts.
6. Audit	Gap Detector	Draft → Gap Loop / Next	Audits evidence coverage; loops back to Searcher if gaps remain.
7. Cite	Citation Mapper	Draft → `cited_report`	Maps source URLs to bracketed inline numeric citations `[N]`.
8. Report	Report Assembly	Draft → `report`	Compiles executive summary, analysis, metrics, and references.
9. Score	Evaluator	Report → `metrics` & `feedback`	Scores 5 quality dimensions & saves lesson learned to vector DB.

3. Models & Key Algorithms

- **LLMs (`Qwen 2.5 3B/7B`, `Phi-3 Mini`)**: Decomposing queries, section synthesis, long-context writing, and evaluator reflection via local Ollama inference.
- **Embedding Model (`nomic-embed-text`)**: 768-dimensional dense vector embeddings for semantic chunk matching and memory retrieval.
- **Cosine Similarity Vector Matching**: Evaluates similarity between sub-question embedding vector v_Q and text chunk embedding v_C .
- **Domain Credibility Tiering**: Assigns weight multipliers (.edu/.gov: 1.0, academic preprints: 0.90, refs: 0.80, blogs: 0.65, spam: 0.0).

- **Content Fingerprinting:** Combines normalized URL + length + prefix SHA-256 hash to eliminate mirror sites and duplicate text.

4. Technical Project Expo & Viva Voce Q&A;

Q1: Why use a Multi-Agent Architecture (LangGraph) instead of a single LLM prompt?

Single LLM calls suffer from context window degradation, parametric hallucinations, and lack multi-stage verification. LangGraph structures processing into specialized micro-agents (Planner, Searcher, Filter, Synthesis, Evaluator), giving strict deterministic state control and verification loops.

Q2: Why choose LangGraph over AutoGen or CrewAI?

AutoGen and CrewAI rely on unpredictable, open-ended conversational agent loops that risk infinite loops and high latency. LangGraph uses an explicit Stateful DAG Graph with typed schemas (`AgentState`), checkpointing (`MemorySaver`), and precise reducer functions (`operator.add`).

Q3: How does the system achieve real-time streaming to the UI?

Via Server-Sent Events (SSE) over HTTP (`StreamingResponse`). Unlike polling (high overhead) or WebSockets (complex bidirectional handshake), SSE allows lightweight, unidirectional streaming of telemetry logs and node state changes to the Next.js client.

Q4: How are hallucinations prevented during report generation?

Through three layers: 1) Strict RAG context grounding (synthesis nodes only write using verified scored snippets); 2) Mandatory Citation Mapping requiring every claim to link to inline `[N]` references; 3) Evaluator Node scoring citation accuracy and pruning ungrounded text.

Q5: What is the 3-Layer Fallback Strategy for web scraping?

1) Fast direct HTTP request + BeautifulSoup HTML parser; 2) Jina Reader Markdown API (`r.jina.ai`) if anti-bot or JS blocks direct extraction; 3) Headless Playwright Chromium browser rendering for SPA pages.

Q6: How does REX learn and improve over time?

At the end of every run, the Evaluator Node extracts a 'Lesson Learned' about search/synthesis performance. This reflection is embedded and saved in Supabase Vector DB. Future runs query this memory index to refine sub-question planning and source selection.